IF431: Algorithms and Complexity

Topic 10: Space and Time Trade-Offs

Moeljono Widjaja, Ph.D.
Marlinda Vasty Overbeek, S.Kom, M.Kom.

Universitas Multimedia Nusantara
Jl. Scientia Boulevard, Gading Serpong
Tangerang, Banten-15811 Indonesia

Even Semester 2025-2026

Expected Learning Outcomes (ELO):

ELO 02 Problem Solving and Critical Thinking: Students should be able to analyze complex problems, evaluate alternatives critically, and develop effective, innovative solutions using logical reasoning.

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

Course Sub Learning Outcomes (Sub-CLO):

Sub-CLO0221: Students are able to apply algorithm analysis and design in relation to its implementation in computer programs. (C3)

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

Review:

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

Transformation Strategy

Make the problem is easier to understand

Problem's instance → Instance Simplification or Representation Change or Problem Reduction → Solution

transform **conquer**

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

Transform-and-Conquer

- Transformation to a simpler or more convenient instance of the same problem—we call it ***instance simplification***.
- Transformation to a different representation of the same instance—we call it ***representation change***.
- Transformation to an instance of a different problem for which an algorithm is already available—we call it ***problem reduction***.

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

Representation Change

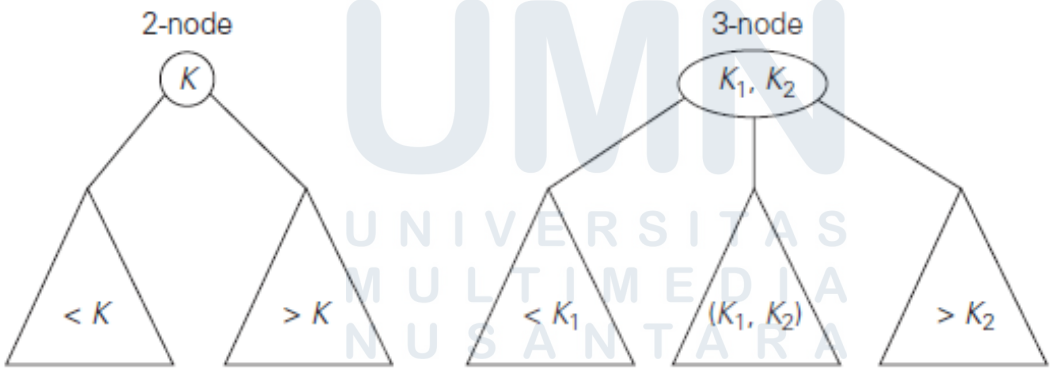
- **2-3 Trees**

  Allow **more than one key** in the same node.

All leaves must be on the same level.
A 2-3 tree is always **perfectly height-balanced**: the length of a path from the root to a leaf is the same for every leaf.

a. **2-node** contains a single key K and has two children (left and right).

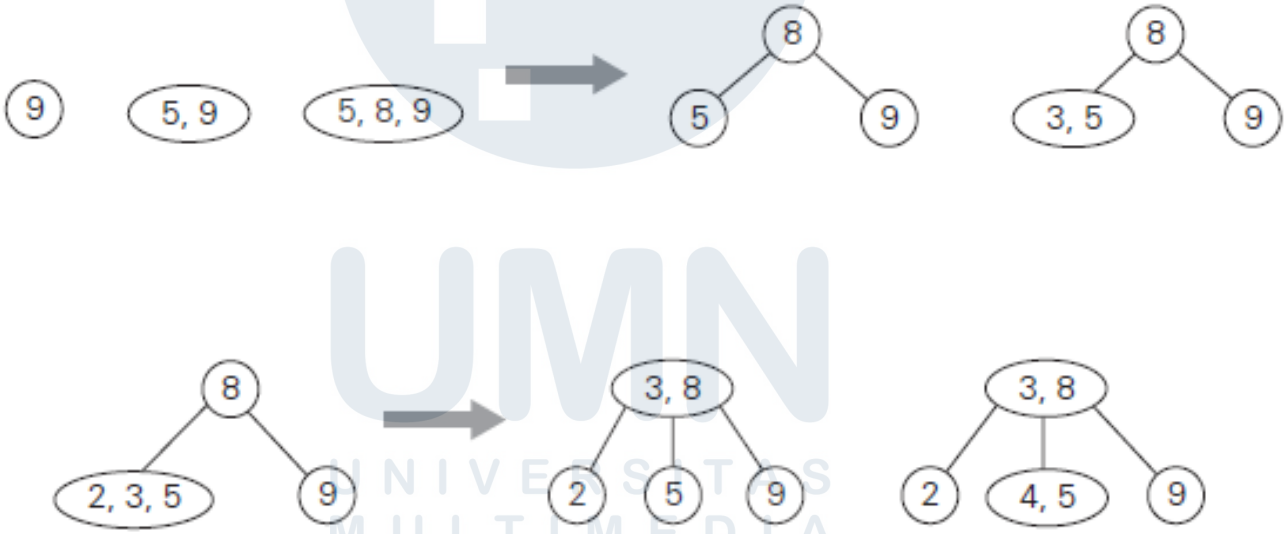
b. **3-node** contains two ordered keys K_1 and K_2 ($K_1 < K_2$) and has 3 children.



UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

Example

Construction of a 2-3 tree for the list 9, 5, 8, 3, 2, 4, 7.



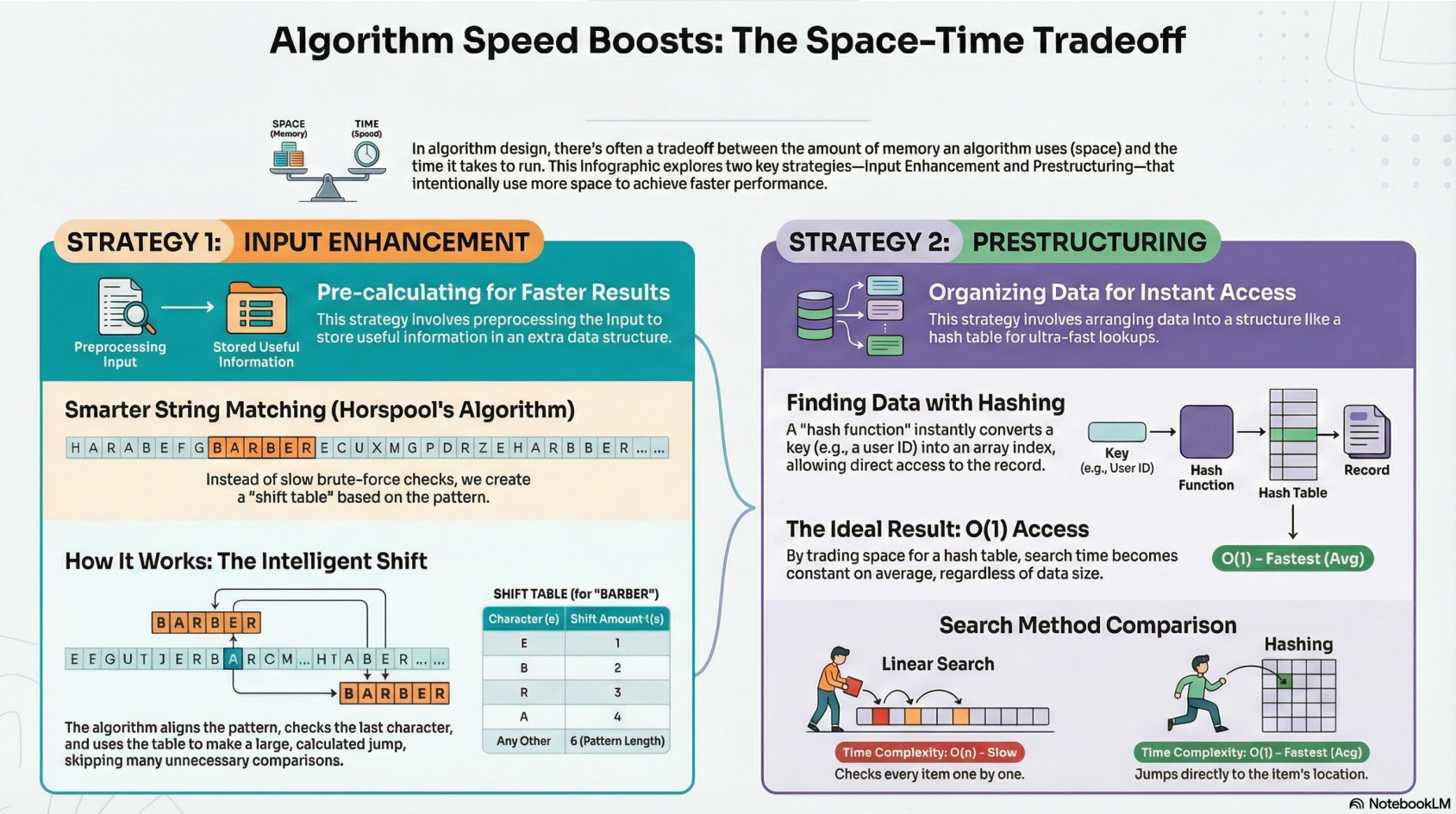
UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

Example

Construction of a 2-3 tree for the list 9, 5, 8, 3, 2, 4, 7.

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

Info Graphic



Outline

1. Space and Time Tradeoffs Concepts
2. Input Enhancement: Sorting by Counting
3. Input Enhancement: String Matching
4. Prestructuring: Hashing
5. Exercises

UMN UNIVERSITAS MULTIMEDIA NUSANTARA

Space and Time Tradeoffs Concepts

Space and Time Tradeoffs

Interplay between time and space in algorithm design

- **Input Enhancement** – preprocess the input (or its part) to **store some info** to be used later in solving the problem.
  a. Sorting by counting
  b. Horspool and Boyer-Moore Algorithm
- **Prestructuring** – preprocess the input to make **accessing its element easier**.
  a. Hashing
  b. Indexing schemes (B-trees)

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

Input Enhancement: Sorting by Counting

Comparison-Counting Sort: Example

To count, for each element of a list to be sorted, the total number of elements smaller than this element and record the results in a table. These numbers will indicate the positions of the elements in the sorted list: e.g., if the count is 10 for some element, it should be in the 10th position (if we start counting with 0) in the sorted array

Array *A*[0..5]		62	31	84	96	19	47
Initially	*Count* []	0	0	0	0	0	0
After pass $i = 0$	*Count* []	3	0	1	1	0	0
After pass $i = 1$	*Count* []		1	2	2	0	1
After pass $i = 2$	*Count* []			4	3	0	1
After pass $i = 3$	*Count* []				5	0	1
After pass $i = 4$	*Count* []					0	2
Final state	*Count* []	3	1	4	5	0	2
Array *S*[0..5]		19	31	47	62	84	96

UNIVERSITAS MULTIMEDIA NUSANTARA

Comparison-Counting Sort

```
ALGORITHM  ComparisonCountingSort(A[0..n − 1])
    //Sorts an array by comparison counting
    //Input: An array A[0..n − 1] of orderable elements
    //Output: Array S[0..n − 1] of A's elements sorted in nondecreasing order
    for i ← 0 to n − 1 do Count[i] ← 0
    for i ← 0 to n − 2 do
        for j ← i + 1 to n − 1 do
            if A[i] < A[j]
                Count[j] ← Count[j] + 1
            else Count[i] ← Count[i] + 1
    for i ← 0 to n − 1 do S[Count[i]] ← A[i]
    return S
```

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

Comparison-Counting Sort

- The number of comparisons:

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1] = \sum_{i=0}^{n-2} (n-1-i) = \frac{n(n-1)}{2}$$

- The algorithm makes the same number of key comparisons as selection sort
- In addition, uses a linear amount of extra space
- On the positive side, the algorithm makes the minimum number of key moves, placing each of them directly in their final position in a sorted array
- This algorithm works productively in a situation in which elements to be sorted belong to a known small set of values

UNIVERSITAS MUHAMMADIYAH MALANG

Input Enhancement: String Matching

String Matching: Revisit

- Given a string of *n* characters called the text and a string of *m* characters ($m \le n$) called the pattern, find substring of the text that <u>matches</u> the pattern

$$
\begin{array}{ccccccccccc}
t_0 & \cdots & t_i & \cdots & t_{i+j} & \cdots & t_{i+m-1} & & \cdots & t_{n-1} & \text{text } T \\
 & & \updownarrow & & \updownarrow & & \updownarrow & & & & \\
 & & p_0 & \cdots & p_j & \cdots & p_{m-1} & \text{pattern } P & & &
\end{array}
$$

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

Brute Force: Revisit

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Text	N	O	B	O	D	Y	_	N	O	T	I	C	E	D	_	H	I	M
Pattern matching	N	O	T															
		N	O	T														
			N	O	T													
				N	O	T												
					N	O	T											
						N	O	T										
							N	O	T									
								N	O	T								

Results → i = 7

Shift the pattern one position to the right for the next trial

Can we do it more efficiently?

UNIVERSITAS MULTIMEDIA NUSANTARA

How far to shift?

1. Case 1

S:	S_0	.	.	.	.	.	.	.	S	.	.	.	.	.	.	.	S_{n-1}
P:				B	A	R	B	E	R								
										B	A	R	B	E	R		

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

How far to shift?

2. Case 2

S:	S_0	.	.	.	.	.	.	.	B	.	.	.	.	.	.	.	S_{n-1}
P:				B	A	R	B	E	R								
						B	A	R	B	E	R						

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

How far to shift?

3. Case 3

S:	S_0	.	.	.	.	.	M	E	R	.	.	.	.	.	.	.	S_{n-1}
P:				B	A	R	B	E	R								
										B	A	R	B	E	R		

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

How far to shift?

4. Case 4

S:	S_0	.	.	.	.	.	.	A	R	.	.	.	.	.	.	.	S_{n-1}
P:				B	A	R	B	E	R								
							B	A	R	B	E	R					

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

Horspool's Algorithm

A simplified version of Boyer-Moore algorithm:

Extra space

- Preprocesses pattern to generate a shift table that determines how much to shift the pattern when a mismatch occurs.
- Always makes a shift based on the text's character ***c*** aligned with the last character in the pattern according to the shift table's entry for ***c***.

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

Shift Table

- We can precompute shift sizes and store them in a table. The table will be indexed by all possible characters that can be encountered in a text, including, for natural language texts, the space, punctuation symbols, and other special characters. The table's entries will indicate the shift sizes computed by the formula

$$t(c) = \begin{cases} \text{the pattern's length } m, \\ \text{if } c \text{ is not among the first } m-1 \text{ characters of the pattern;} \\ \\ \text{the distance from the rightmost } c \text{ among the first } m-1 \text{ characters} \\ \text{of the pattern to its last character, otherwise.} \end{cases}$$

For example, for the pattern BARBER, all the table's entries will be equal to 6, except for the entries for E, B, R, and A, which will be 1, 2, 3, and 4, respectively.

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

Shift Table

ALGORITHM *ShiftTable*($P[0..m-1]$)

```
//Fills the shift table used by Horspool's and Boyer-Moore algorithms
//Input: Pattern P[0..m − 1] and an alphabet of possible characters
//Output: Table[0..size − 1] indexed by the alphabet's characters and
//        filled with shift sizes computed by formula (7.1)
for i ← 0 to size − 1 do Table[i] ← m
for j ← 0 to m − 2 do Table[P[j]] ← m − 1 − j
return Table
```

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

Example

- Shift table for BARBER in a text that comprises English letters and spaces (denoted by underscores).

Character *c*	**A**	**B**	**C**	**D**	**E**	**F**	.	.	.	**R**	.	.	.	.	**Z**	_
Shift t(c)	4	2	6	6	1	6	6	6	6	3	6	6	6	6	6	6

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

Horspool's Algorithm

```
ALGORITHM  HorspoolMatching(P[0..m − 1], T[0..n − 1])
    //Implements Horspool's algorithm for string matching
    //Input: Pattern P[0..m − 1] and text T[0..n − 1]
    //Output: The index of the left end of the first matching substring
    //          or −1 if there are no matches
    ShiftTable(P[0..m − 1])        //generate Table of shifts
    i ← m − 1                      //position of the pattern's right end
    while i ≤ n − 1 do
        k ← 0                      //number of matched characters
        while k ≤ m − 1 and P[m − 1 − k] = T[i − k] do
            k ← k + 1
        if k = m
            return i − m + 1
        else  i ← i + Table[T[i]]
    return −1
```

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

Horspool's Algorithm

Example

Shift 4 Shift 1 Shift 6 Shift 2 Shift 3

J	I	M	_	S	A	W	_	M	E	_	I	N	_	A	_	B	A	R	B	E	R	S	H	O	P
B	A	R	B	E	R																				
				B	A	R	B	E	R																
					B	A	R	B	E	R															
											B	A	R	B	E	R									
													B	A	R	B	E	R							
																B	A	R	B	E	R				

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

Boyer-Moore Algorithm

Based on same two ideas:

1. Comparing pattern characters to text from right to left
2. Precomputing **shift sizes in <u>two tables</u>**
   - **bad-symbol table** $t(c)$ (same as Horspool table), but in use
     - Shift uses text character at mismatch (not the last one as Horspool's)
     - Shift uses length of matching suffix
   - **good-suffix table** d_2 indicates how much to shift based on **matched** part (suffix) of the pattern.
   - Shift the pattern to the right by the number of positions computed by the formula:

$$d = \begin{cases} d_1 & \text{if } k = 0, \\ \max\{d_1, d_2\} & \text{if } k > 0, \end{cases}$$

where $d_1 = \max\{t_1(c) - k, 1\}$.

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

Boyer-Moore Algorithm

Bad-symbol Shift

- For Case 1 and 2, Boyer-Moore acts like Horspool's.
- Otherwise, if the rightmost character of the pattern does match, Boyer-Moore compares preceding characters right to left until either all pattern's characters match or a mismatch on text's character c is encountered after $k > 0$ matches.

S:	S_0	.	.	.	.	.	S	E	R	.	.	.	.	.	.	.	S_{n-1}
P:				B	A	R	B	E	R								

bad-symbol shift $d_1 = \max\{t_1(c) - k, 1\}$

UMN UNIVERSITAS MULTIMEDIA NUSANTARA

Boyer-Moore Algorithm

Good-suffix Shift

- Good-suffix shift d_2 is applied after $0 < k < m$ last characters were matched.
- $d_2(k)$ = the distance between matched suffix of size k and its rightmost occurrence in the pattern that is not preceded by the same character as the suffix.

k	Pattern P	d_2
1	ABCBAB	2
2	ABCBAB	4
3	ABCBAB	4
4	ABCBAB	4
5	ABCBAB	4

UMN UNIVERSITAS MULTIMEDIA NUSANTARA

Boyer-Moore Algorithm

B	E	S	S	_	K	N	E	W	_	A	B	O	U	T	_	B	A	O	B	A	B	S
B	A	O	B	A	B																	

Bad-symbol Shift Table

c	A	B	C	D	E	F	.	.	.	O	.	.	.	.	Z	_
$t_1(c)$	1	2	6	6	6	6	6	6	6	3	6	6	6	6	6	6

k	pattern	d_2
1	BAOBAB	2
2	BAOBAB	5
3	BAOBAB	5
4	BAOBAB	5
5	BAOBAB	5

Good-suffix Shift Table

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

Boyer-Moore Algorithm: Summary

The Boyer-Moore algorithm

Step 1 For a given pattern and the alphabet used in both the pattern and the text, construct the bad-symbol shift table as described earlier.

Step 2 Using the pattern, construct the good-suffix shift table as described earlier.

Step 3 Align the pattern against the beginning of the text.

Step 4 Repeat the following step until either a matching substring is found or the pattern reaches beyond the last character of the text. Starting with the last character in the pattern, compare the corresponding characters in the pattern and the text until either all m character pairs are matched (then stop) or a mismatching pair is encountered after $k \geq 0$ character pairs are matched successfully. In the latter case, retrieve the entry $t_1(c)$ from the c's column of the bad-symbol table where c is the text's mismatched character. If $k > 0$, also retrieve the corresponding d_2 entry from the good-suffix table. Shift the pattern to the right by the number of positions computed by the formula

$$d = \begin{cases} d_1 & \text{if } k = 0, \\ \max\{d_1, d_2\} & \text{if } k > 0, \end{cases}$$

where $d_1 = \max\{t_1(c) - k, 1\}$.

UNIVERSITAS MULTIMEDIA NUSANTARA

Boyer-Moore Algorithm

mismatch

B	E	S	S	_	K	N	E	W	_	A	B	O	U	T	_	B	A	O	B	A	B	S
B	A	O	B	A	B																	

$d_1 = t_1(K) - 0 = 6$

mismatch Match, *k*=2

B	E	S	S	_	K	N	E	W	_	A	B	O	U	T	_	B	A	O	B	A	B	S
						B	A	O	B	A	B											

$d_1 = t_1(_) - 2 = 4$
$d_2 = 5$
$d = \max(4,5) = 5$

Match, *k*=1

B	E	S	S	_	K	N	E	W	_	A	B	O	U	T	_	B	A	O	B	A	B	S
											B	A	O	B	A	B						

$d_1 = t_1(_) - 1 = 5$
$d_2 = 2$
$d = \max(5,2) = 5$

UNIVERSITAS MULTIMEDIA NUSANTARA

Prestructuring: Hashing

Prestructuring

- Linear search time in array is *O(n)*
- Binary search time in array is *O(log n)*
- Ideally, we want constant *O(1)* in the average case → Direct Access
- Direct Address Table is a data structure that has the capability of mapping records to their corresponding keys using arrays.
- In direct address tables, records are placed using their key values directly as indexes.
  - Prestructuring → Hashing (trade-off space vs time)
- They facilitate fast searching, insertion and deletion operations.

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

Hashing

- **Hashing** is based on the idea of distributing keys among a one-dimensional array H[0..m−1] called a **hash table**. The distribution is done by computing, for each of the keys, the value of some predefined function h called the **hash function**.
- This function assigns an integer between 0 and $m-1$, called the **hash address**, to a key.

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

HASH TABLE

CODE	CODE % 23
61	15
8	8
46	0
35	12
25	2
86	17
49	3
44	21
19	19
30	7
33	10
5	5
64	18
50	4

INDEX	CODE	NAME	JOB
0	46	John McCarthy	Artificial intelligence
1			
2	25	Donald E. Knuth	Art of programming
3	49	C.A.R. Hoare	Design of prog. language
4	50	Raj Reddy	Large scale AI systems
5	5	John Hopcroft	Analysis of algorithms
6			
7	30	Dennis M. Ritchie	Generic OS theory
8	8	Marvin Minsky	Artificial intelligence
9			
10	33	Niklaus Wirth	Pascal
11			
12	35	E.W. Dijkstra	Programming language
13			
14			
15	61	Richard Hamming	Error correcting codes
16			
17	86	Dana S. Scott	Finite automata
18	64	Amir Pnueli	Temporal logic
19	19	Ken Thompson	UNIX OS
20			
21	44	Edgar F. Codd	DBMS
22			

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

SEARCH HASH TABLE

Key? 30
Index = ?
30 % 23 = [7]
Dennis M. Ritchie
Generic OS Theory

Key? 60
Index = ?
60 % 23 = [14]
Not found

INDEX	CODE	NAME	JOB
0	46	John McCarthy	Artificial intelligence
1			
2	25	Donald E. Knuth	Art of programming
3	49	C.A.R. Hoare	Design of prog. language
4	50	Raj Reddy	Large scale AI systems
5	5	John Hopcroft	Analysis of algorithms
6			
7	30	Dennis M. Ritchie	Generic OS theory
8	8	Marvin Minsky	Artificial intelligence
9			
10	33	Niklaus Wirth	Pascal
11			
12	35	E.W. Dijkstra	Programming language
13			
14			
15	61	Richard Hamming	Error correcting codes
16			
17	86	Dana S. Scott	Finite automata
18	64	Amir Pnueli	Temporal logic
19	19	Ken Thompson	UNIX OS
20			
21	44	Edgar F. Codd	DBMS
22			

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

Hashing

- Hash Table
- Hash Function
  - Digit Selection
  - Folding
  - Modular Arithmetic
- Collision
  - Linear Probing
  - Rehashing
  - Double hashing
  - Separate chaining

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

Exercises

EXERCISES

1. Finish array below with counting sort

   6 – 33 – 12 – 24 – 27 – 9 – 20 - 65

2. Make the simulation process of Horspool's for:

   Text: **HERE IS A SIMPLE EXAMPLE OF STRING MATCHING ALGORITHM**

   Pattern: **MATCHING**

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

Next Week

Dynamic Programming

- Dynamic Programming concepts
- Motivating Examples (Non-Optimization Problems)
- Optimization Problems

UMN UNIVERSITAS MULTIMEDIA NUSANTARA

Vision and Mission

Vision

Becoming a leading Informatics Undergraduate Study Program that produces graduates with international insight who are competent in the field of computer science, have an entrepreneurial spirit, and have noble character.

Mission

1. Organising learning with the best technology and curriculum and supported by professional teaching staff.
2. Carry out research activities in the field of informatics to advance information science and technology.
3. Carry out community service activities based on information technology and science in the context of practising informatics science and technology society.